

Machine Health

Guia de estudio para la exposicion — Hackathon IA, Facultad de Ingenieria UNaM. Equipo Acosta, Borges, Pelinski.

Para que sirve este documento

Explica de cero los dos notebooks que hay que presentar: **Intento3**, que consiguio el mejor puntaje del equipo (**48,1150**), y **base4.5**, que es el estado actual del trabajo. No supone conocimiento previo: arranca por el problema, sigue por la metrica y recién despues entra al codigo.

Todos los numeros que aparecen salen de las salidas guardadas en los notebooks. Si un numero no estaba, no se inventa: se dice que falta.

Como estudiarlo: las partes 1 y 2 son las que hay que entender de verdad; las partes 3 y 4 son el recorrido de cada notebook; la parte 6 es un guion de exposicion y la parte 7 anticipa preguntas.

Parte 1 — El problema

El contexto

Una planta industrial opera un parque de **maquinas rotativas**: motores electricos acoplados a bombas hidraulicas. Cada maquina tiene sensores de vibracion, corriente, tension, temperatura, presion y caudal. Las senales se agrupan en **ventanas temporales** de dos segundos, y de cada ventana se calculan descriptores estadisticos y espectrales.

El sistema de mantenimiento tiene que mirar esos numeros y decidir **que hacer con la maquina**. Ese es el punto que hay que tener claro desde el principio: el problema no termina en clasificar, termina en **decidir una accion**, y la metrica mide el costo economico de esa decision.

Las 13 fallas y las 4 familias

Se consideran 13 modos de falla, agrupados en cuatro familias. La familia importa muchisimo, por razones que se ven en la Parte 2.

Familia	ID	Descripcion
Mecanica	F01	Desbalance de rotor
	F02	Desalineacion
	F03	Rodamiento, pista exterior
	F04	Rodamiento, pista interior
	F05	Elemento rodante (BSF)
	F06	Lubricacion deficiente
Estructural	F07	Perdida de rigidez / tornilleria
Electrica	F08	Desequilibrio de tension / fases
	F09	Perdida de fase / intermitencia
	F10	Barras rotoricas
	F11	Cortocircuito entre espiras
	F12	Cavitacion
Hidraulica	F13	Obstruccion / fuga hidraulica

Seis fallas son mecanicas, una estructural, cuatro electricas y dos hidraulicas.

Los datos

Tabla	Filas	Columnas	Tiene etiquetas
train_sup	1750	77	Si: label_F01..F13 y severity_F01..F13
test	1848	49	No. Es lo que hay que predecir
train_unlabeled	434	49	No
raw_signal_index	4032	3	Mapea cada ventana a su archivo NPZ

Hay **46 variables tabulares** utilizables, con el patron <sensor>__<metrica>: por ejemplo acc_radial_a__rms (valor eficaz de la vibracion radial), acc_radial_a__1x (amplitud a la frecuencia de giro), voltage_a__rms, temp_winding__mean, flow_mean, rpm_mean.

Ademas existen las **senales crudas** en archivos NPZ, un archivo por sesion. Son tres canales: acelerometro a 20 kHz, corriente a 10 kHz y presion a 200 Hz. Cada ventana dura **2,00 segundos**, o sea 40.000 muestras de aceleracion, 20.000 de corriente y 400 de presion. El paquete completo pesa 664 MB. Su uso es opcional segun la consigna.

Tres identificadores, y por que importan

- **window_id**: identifica cada ventana de dos segundos. Es la clave de la entrega.
- **session_id**: identifica una sesion de captura. Cada sesion tiene exactamente 7 ventanas.
- **machine_id**: identifica la maquina fisica. Varias sesiones pueden ser de la misma maquina.

Los tres hechos estructurales que ordenan todo el trabajo

1. No es multilabel, es multiclase de 14 estados. La consigna dice que una maquina puede tener varias fallas a la vez y pide 13 probabilidades independientes. Pero en las 1750 filas etiquetadas **no hay ni una sola ventana con dos fallas**: 217 estan sanas y 1533 tienen exactamente una. La columna is_combo vale cero en todas. Por eso se modela con un *softmax* de 14 estados (sano + F01..F13) y no con 13 clasificadores binarios sueltos.

2. La etiqueta es constante dentro de cada sesion. Las 7 ventanas de una sesion comparten la misma etiqueta, en las 250 sesiones. Eso significa que el **tamano efectivo de la muestra son 250 casos, no 1750**. Es siete veces menos de lo que sugiere el numero de filas, y hace que el riesgo de sobreajuste sea mucho mayor.

3. Train y test no comparten ninguna maquina. 26 maquinas en entrenamiento, 22 en test, interseccion vacia. Y ademas operan en otro punto de trabajo: las rpm medias pasan de 1742 a 2052 y la presion diferencial de 0,51 a 2,61, cinco veces mas. El modelo tiene que generalizar a maquinas nuevas y a un regimen nuevo.

Prevalencia: cuantos casos hay de cada cosa

Estado	Casos	Prevalencia	Familia
sano	217	12,4 %	—
F01	168	9,6 %	mecanica
F02	168	9,6 %	mecanica
F03	161	9,2 %	mecanica
F04	98	5,6 %	mecanica
F05	112	6,4 %	mecanica
F06	105	6,0 %	mecanica
F07	91	5,2 %	estructural
F08	112	6,4 %	electronica

F09	84	4,8 %	electrica
F10	84	4,8 %	electrica
F11	98	5,6 %	electrica
F12	161	9,2 %	hidraulica
F13	91	5,2 %	hidraulica

El desbalance es moderado: de 4,8 % a 9,6 % por falla. No hace falta remuestrear. De las ventanas con falla, el **57,5 % son severas** (severidad mayor a 0,5), dato que importa porque el costo de no detectar una falla severa triplica al de una leve.

Que hay que entregar

Un CSV con una fila por ventana del test y 14 columnas: `window_id`, `F01`, `F02`, ..., `F13`. Cada columna es la **probabilidad estimada** de esa falla, un numero entre 0 y 1. No se entregan clases, se entregan probabilidades. Son 1848 filas.

Parte 2 — La metrica, explicada de cero

Esta es la parte que mas conviene dominar. Casi todas las decisiones del proyecto se explican mirando como esta armada la metrica.

El puntaje final tiene tres componentes

$$\text{final_score} = 0,70 \times \text{cost_score} + 0,20 \times \text{prob_score} + 0,10 \times \text{diag_score}$$

Va de 0 a 100 y **mayor es mejor**. Cada componente mide algo distinto:

Componente	Peso	Que mide	Formula
cost_score	70 %	cuanto dinero ahorran las decisiones	$100 \times (1 - \text{costo_medio} / \text{costo_de_no_hacer_nada})$
prob_score	20 %	que tan calibradas estan las probabilidades	$100 \times (1 - \text{Brier} / 0,25)$
diag_score	10 %	si detecta las fallas correctas	$100 \times \text{F1_macro con umbral } 0,5$

Como funciona cost_score: el sistema decide una accion

Aca esta lo importante y lo que mas cuesta explicar. **Nosotros no elegimos la accion**. Entregamos 13 probabilidades y el servidor corre su propia funcion `choose_action`, que elige entre cuatro acciones la de menor costo esperado *segun nuestras probabilidades*:

Accion	Que significa	Costo real
MONITOR	no intervenir	0 si esta sana; 5 si hay falla leve; 15 si hay falla severa
INSPECT familia	inspeccionar una familia	4 si acierta la familia; $4 + 8 = 12$ si se equivoca
DERATE	operar a carga reducida	$7 + 35 \%$ del falso negativo
STOP	detener la maquina	12, sin falso negativo

Despues el servidor mira la verdad y cobra el costo real de la accion elegida. El `cost_score` compara ese costo contra el de no hacer nada nunca (MONITOR siempre), que da **9,42** en promedio.

Consecuencia numero uno: el 70 % del puntaje se juega en la FAMILIA

Mirando la tabla de costos con una sola familia verdadera presente:

Sana MONITOR 0 | INSPECT correcta 4 | INSPECT incorrecta 4

Falla leve MONITOR 5 | INSPECT correcta 4 | INSPECT incorrecta 12

Falla sev. MONITOR 15 | INSPECT correcta 4 | INSPECT incorrecta 12

Confundir F03 con F04 no cuesta un centavo: las dos son mecanicas, la accion INSPECT mecanica es la misma y cuesta 4 igual. Lo unico que cambia el costo es acertar la **familia** — cinco categorias: sana, mecanica, estructural, electrica, hidraulica — y no la falla exacta. Distinguir fallas dentro de una familia solo suma al `diag_score`, que pesa 10 %.

Consecuencia numero dos: el maximo alcanzable es 73,96, no 100

Aun con prediccion perfecta hay que **pagar la inspeccion de 4 dolares** en cada ventana con falla. Como el 87,6 % de las ventanas tiene falla, el costo minimo posible es $0,876 \times 4 = 3,50$ contra los 9,42 de no hacer nada.

$$\text{cost_score maximo} = 100 \times (1 - 3,50/9,42) = 62,80$$

Y con `prob_score` y `diag_score` en 100, el final maximo es $0,70 \times 62,80 + 0,20 \times 100 + 0,10 \times 100 = 73,96$.

Comparar nuestro puntaje contra 100 es enganoso. Estar en 48 sobre un techo de 74 es estar en el 65 % de lo alcanzable, no en el 48 %.

El baseline de la catedra no es pasivo (dato para lucirse)

El notebook de ejemplo que reparte la catedra dice que, con probabilidades bajas, la accion optima es MONITOR. **Con sus propias prevalencias eso es falso**, y se puede demostrar en tres lineas:

Suma de todas las probabilidades:	$S = 0,876$	
Suma de las mecanicas:	$S_{mecanica} = 0,464$	
Costo esperado de MONITOR	$= 10 \times 0,876$	$= 8,76$
Costo esperado de INSPECT mec.	$= 4 + 8 \times (0,876 - 0,464)$	$= 7,30$ <-- gana
Costo esperado de STOP	$=$	$12,00$

La regla elige **INSPECT mecanica en las 1848 ventanas**. Lo verificamos corriendo la funcion oficial: devuelve exactamente eso. El baseline no es un modelo pasivo, es uno que **siempre inspecciona la familia mas frecuente**. Por eso saca 29 y no 15, y por eso para superarlo hay que acertar la familia mas del 46 % de las veces. Es un hallazgo defendible y muestra que leimos la metrica en serio.

prob_score: por que conviene entregar probabilidades honestas

El Brier es el promedio de $(\text{probabilidad predicha} - \text{verdad})^2$ sobre las 1848 ventanas y las 13 etiquetas. Un Brier de 0 es perfecto y 0,25 es el peor caso. Como pesa 20 %, **inflar o afilar las probabilidades para forzar acciones sale caro**: lo poco que se gana en costo se pierde en calibracion. Lo medimos: afilar las probabilidades elevandolas a una potencia empeora el puntaje final.

diag_score: el umbral fijo de 0,5

Es el F1 macro, calculado binarizando cada probabilidad en 0,5. **Macro** significa que cada una de las 13 fallas pesa igual, incluidas las raras. El umbral es fijo y no lo elegimos nosotros: si el modelo es prudente y nunca pasa de 0,5 en una falla poco frecuente, esa falla aporta $F1 = 0$ aunque el ranking sea bueno.

Consecuencia numero tres: no hay nada que ganar manipulando la decision

Como la metrica corre su propia choose_action sobre nuestras probabilidades, uno podria intentar distorsionarlas para forzar la accion que mas conviene. Lo probamos: la regla del servidor ya coincide en el **99,1 %** de las ventanas con la accion optima calculada con nuestro propio modelo. No queda margen. La unica forma de subir el cost_score es **predecir mejor la familia**.

Resumen de la Parte 2 en una frase

El puntaje premia acertar la familia de la falla con probabilidades bien calibradas, tiene un techo real de 73,96, y no se puede hackear la capa de decision. Todo lo que viene despues son intentos de acertar mejor la familia.

Parte 3 — Notebook Intento3: el del mejor puntaje

Archivo: notebooks/eda_y_benchmark__Intento3.ipynb. 62 celdas. Es el que produjo la entrega de **48,1150**, el mejor puntaje del equipo.

Seccion 0 y 1 — Entorno, carga y verificacion

Descarga los dos ZIP desde Drive (el kit de la catedra con `scoring.py`, y los datos), y carga las tablas. Despues verifica lo que no se puede asumir:

```
train (1750, 77) | test (1848, 49)
maquinas -> train 26, test 22, interseccion 0
sesiones -> train 250, test 264, interseccion 0

columnas con nulos: 0          filas duplicadas: 0
columnas constantes: temp_winding__nan_frac, temp_bearing1__nan_frac,
                    temp_bearing2__nan_frac, is_combo
```

Que decir de esto: los datos vienen limpios, no hay nada que curar. El trabajo no esta en la limpieza sino en las variables. Y aparece el primer hallazgo: `is_combo` es una columna constante en cero, que es la punta del hilo del problema de 14 estados.

Seccion 2 — Mitigacion del cambio de distribucion

El problema: como las maquinas de test son otras y giran mas rapido, los **valores absolutos** de los sensores identifican a la maquina mas que al estado de salud. Un modelo entrenado con valores absolutos aprende las 26 maquinas del entrenamiento y no generaliza.

La correccion es expresar cada variable como **desviacion respecto del comportamiento habitual de su propia maquina**:

```
z = (valor - mediana_de_esa_maquina) / (desvio_de_esa_maquina + epsilon)
```

Se usa **mediana y no media** por robustez: las ventanas en falla son justamente las colas de la distribucion de cada maquina, y la media se contaminaria con ellas. Es legitimo aplicarlo al test porque solo usa las columnas de variables y el `machine_id`, nunca las etiquetas.

El efecto, medido con el test de Kolmogorov-Smirnov, que mide cuanto se diferencian dos distribuciones:

```
KS medio train vs test:  crudo = 0,139    ->  z-score = 0,059
variables con KS > 0,2:  crudo = 11       ->  z-score = 0
```

Las once variables claramente desalineadas quedan en cero y el KS medio cae a menos de la mitad. **Se conservan las dos versiones** de cada variable, la original y la normalizada: los valores absolutos siguen teniendo informacion fisica real y los arboles eligen cual usar. De 46 variables se pasa a 92.

Seccion 2.5 — EDA supervisado

Distribuciones, deteccion de valores atipicos, matriz de correlacion y graficos de dispersion. Detecta **64 pares de variables muy correlacionadas** (34 si se miran solo las originales). Es diagnostico: en este notebook todavia no se actua sobre eso.

Feature engineering: las variables fisicas

Este es el aporte central del notebook. La idea: una variable que depende del punto de trabajo no transfiere a un regimen nuevo; un **cociente entre magnitudes que escalan igual**, si. Se construyen 42 variables nuevas, todas adimensionales o relativas:

Bloque

Ejemplos

Que falla ataca

Vibracion	$\text{rms} / (\text{rpm}/1000)^2 \cdot 2x/1x \cdot 1x/\text{rms} \cdot \log(1+\text{curtosis})$	F01 desbalance, F02 desalineacion, F03-F05 rodamientos
Asimetria	$\text{acc_axial}/\text{acc_radial} \cdot \text{radial_b}/\text{radial_a}$	F07 perdida de rigidez
Electricas	$(\text{max-min})/\text{media de tension y corriente} \cdot I_{\text{min}}/I_{\text{media}} \cdot I/\text{rpm}$	F08 desequilibrio, F09 perdida de fase, F10 barras
Termicas	$\text{temp_devanado} - T_{\text{amb}} \cdot \text{temp_rod1} - \text{temp_rod2} \cdot (T_{\text{dev}} - T_{\text{amb}})/\text{rpm}$	F06 lubricacion, F11 cortocircuito entre espiras
Hidraulicas	$\text{delta_p} / (\text{rpm}/1000)^2 \cdot \text{caudal}/\text{rpm} \cdot \text{caudal} \times \text{delta_p} / (V \cdot I)$	F12 cavitacion, F13 obstruccion o fuga

Por que funcionan, en criollo

La energia de vibracion crece con el cuadrado de la velocidad. Si no se divide por $(\text{rpm}/1000)^2$, el modelo confunde *maquina rapida* con *maquina en falla* — que es exactamente la diferencia entre nuestro entrenamiento y el test.

La desalineacion excita el segundo armonico, por eso el cociente $2x/1x$ marca F02; el desbalance concentra energia en el primero, por eso $1x/\text{rms}$ marca F01.

Las temperaturas van siempre contra el ambiente, nunca en valor absoluto, porque el ambiente del test es 3 grados mas calido.

Las bombas siguen leyes de afinidad: la altura escala con rpm^2 y el caudal con rpm , asi que $\text{delta_p}/(\text{rpm}/1000)^2$ y caudal/rpm son adimensionales.

Resultado: 45 originales + 42 fisicas + 87 z-score = **174 columnas**.

Seccion 3 — El esquema de validacion

Se usa StratifiedGroupKFold con 5 particiones, **agrupando por maquina**. Estratificar quiere decir que cada particion tenga la misma proporcion de cada estado; agrupar por maquina quiere decir que ninguna maquina aparezca a los dos lados. Sin eso, el modelo veria en validacion la misma maquina que vio entrenando y el resultado seria una mentira.

```
fold 0: 1407 ajuste / 343 validacion | maquinas de validacion: 5
fold 1: 1386 ajuste / 364 validacion | maquinas de validacion: 5
fold 2: 1358 ajuste / 392 validacion | maquinas de validacion: 6
fold 3: 1407 ajuste / 343 validacion | maquinas de validacion: 5
fold 4: 1442 ajuste / 308 validacion | maquinas de validacion: 5
```

El notebook tambien define una **segunda validacion por cambio de regimen**: ordena las 26 maquinas por rpm, entrena con las 13 lentas (1437 rpm de media) y valida con las 13 rapidas (2057 rpm), que es casi exactamente el regimen del test (2052 rpm). Reproduce la relacion real entre entrenamiento y test.

Y fija las dos referencias sin las cuales ningun numero significa nada:

```
P0 prevalencia (baseline) final = 30,78 cost = 22,55
oraculo (etiquetas exactas) final = 73,96 cost = 62,80
```

Resultados con las 92 columnas (originales + z-score)

Modelo	final	cost	prob	diag	acc familia	sd folds
XGBoost	45,52	36,91	80,02	36,80	0,616	4,68
RegLogistica	45,19	37,69	76,33	35,40	0,615	3,13
RandomForest	44,44	35,94	82,65	27,59	0,605	5,10
CatBoost	43,70	34,61	81,43	31,84	0,587	4,10
LightGBM	43,30	33,97	78,53	38,10	0,588	4,87
ExtraTrees	43,28	35,23	81,93	22,36	0,583	4,57

Los seis modelos caen en 2,3 puntos, un margen mas chico que el desvio entre particiones de casi todos ellos (entre 3,1 y 5,1). Conclusion: **el algoritmo no es la palanca**. Buscar un modelo mejor es tiempo perdido; el limite esta en la informacion que traen las variables.

Pero **ningun modelo gana en las tres componentes**: los boosters (LightGBM, XGBoost) ganan en diag_score porque discriminan mejor; los de bagging (RandomForest, ExtraTrees) ganan en prob_score porque promedian muchos arboles y salen mejor calibrados. Eso es una invitacion al ensamble.

El ensamble: promediar probabilidades

Ensamble (92 columnas)	final	cost	prob	diag	sd folds
LightGBM solo	43,30	33,97	78,53	38,10	4,87
LGB + XGB	43,73	34,46	79,59	36,93	4,82
LGB + XGB + Cat	44,42	35,08	80,87	36,84	4,64
LGB + XGB + Cat + RF	44,66	35,41	81,74	35,25	4,35
LGB + RF + RegLogistica	47,33	39,18	82,10	34,92	3,27

El promedio simple de probabilidades gana en todos los casos. El mejor combina LightGBM, RandomForest y una regresion logistica: **+4,0 sobre el mejor individual**, y ademas con el desvio entre particiones mas bajo de todos.

Por que funciona: la regresion logistica es el peor modelo individual y es la que hace andar el ensamble. Es la unica **lineal**, asi que sus errores estan descorrelacionados de los de los arboles. En un promedio, la

diversidad vale mas que la calidad individual.

Se confirmo con una segunda semilla de particion para descartar que fuera casualidad: los cinco ensambles mejoran en las dos semillas.

El paso decisivo: las variables fisicas

Repitiendo todo con las **174 columnas** en lugar de las 92:

Modelo (174 columnas)	final	cost	prob	diag	macro F1	acc familia
CatBoost	50,86	43,00	83,86	39,85	0,399	0,678
RegLogistica	50,26	42,11	80,26	47,28	0,473	0,669
XGBoost	49,74	41,09	82,69	44,41	0,444	0,653
RandomForest	49,40	42,11	83,94	31,33	0,313	0,659
LightGBM	48,83	39,90	81,49	46,09	0,461	0,645
ExtraTrees	47,09	39,51	83,45	27,39	0,274	0,643

Ensamble (174 columnas)	final	cost	prob	diag	por particion				
LGB + RF + RegLogistica	51,98	43,51	84,45	46,30	53,9	48,6	52,8	59,2	42,9
LGB + XGB + Cat + RF	50,72	42,44	84,08	41,92	51,6	46,9	52,3	58,7	41,7
LGB + XGB + Cat	50,49	42,00	83,58	43,70	51,7	45,9	52,1	58,3	41,7
LGB + XGB	49,64	40,94	82,45	44,89	51,5	43,8	49,4	58,9	41,9
LightGBM solo	48,83	39,90	81,49	46,09	51,9	43,1	48,8	58,3	39,5

El resultado que hay que saber de memoria

Agregar las variables fisicas sube el ensamble de **47,33 a 51,98** en validacion cruzada: **+4,65 puntos**. La exactitud de familia sube de 0,615 a 0,678.

Ese modelo — **LightGBM + RandomForest + regresion logistica sobre 174 columnas** — es el que se envio, y el servidor devolvio **48,1150**.

La diferencia entre 51,98 de validacion y 48,11 del servidor es normal y esperable: la validacion cruzada por maquina es optimista porque valida con maquinas del mismo regimen, y el test tiene regimen nuevo.

Que se envio, concretamente

- **Variables:** 45 originales + 42 fisicas + 87 z-score por maquina = 174.
- **Modelo:** softmax de 14 estados, promedio simple de LightGBM, RandomForest y regresion logistica.
- **Validacion:** StratifiedGroupKFold de 5 particiones agrupado por maquina.
- **Entrega:** se descarta la columna del estado sano y se entregan las 13 restantes como probabilidades.
- **Puntaje del servidor:** **48,1150**. Referencias: baseline 30,78 en validacion, techo del oraculo 73,96.

Parte 4 — Notebook base4.5: el estado actual

Archivo: eda_y_benchmark__base4.5.ipynb. 89 celdas.

Lo primero que hay que tener claro al presentarlo

base4.5 no cambia el modelo. Contiene exactamente el mismo pipeline que Intento3 y reproduce los mismos numeros: las 174 columnas, el ensamble de LightGBM + RandomForest + regresion logistica, y el mismo 51,98 en validacion.

Lo que agrega son dos cosas: un **EDA mucho mas profundo** que justifica por que las variables son las que son, y el **arranque del trabajo con las senales crudas**. Si alguien pregunta cuanto subio el puntaje base4.5, la respuesta honesta es que no subio: es trabajo de analisis y de preparacion, no de modelo.

Lo que agrega, seccion por seccion

Catalogacion de variables

Etiqueta las 174 columnas en tres grupos: 45 originales, 42 fisicas y 87 z-score. Suena trivial pero es lo que permite todos los analisis que siguen, porque deja preguntar cosas como *las fisicas, ¿aportan algo que las originales no tengan?*

Calidad estadistica

Para cada variable calcula media, desvio, asimetria, coeficiente de variacion, cuartiles, porcentaje de valores atipicos por el criterio del rango intercuartil, y cardinalidad. El resumen:

```
features con missing      : 0
casi constantes           : 22
discretas / baja cardinalidad : 22
muy sesgadas              : 54
con muchos outliers IQR   : 78
```

Como leerlo: cero faltantes confirma que no hay que imputar. Pero 78 de 174 variables tienen muchos valores atipicos y 54 estan muy sesgadas. Eso no es un defecto de los datos: en deteccion de fallas **el valor atipico es justamente la senal**. Es un argumento a favor de usar arboles, que no se molestan con distribuciones asimetricas, y en contra de modelos que asumen normalidad.

Analisis de redundancia

```
Pares |corr| > 0,90: 90
Pares |corr| > 0,95: 57
```

Ademas, para cada variable fisica busca cual es la original mas parecida, por correlacion de Pearson y de Spearman. Sirve para detectar variables fisicas que son una copia disfrazada de una original y no aportan nada nuevo.

Un resultado contraintuitivo que conviene tener a mano

Con toda esa redundancia identificada, lo natural seria limpiarla. **Se probó y empeora el puntaje.** Quitar las 12 columnas mas redundantes hizo bajar el resultado de una prueba posterior de 45,44 a 44,45.

La razon: RandomForest y ExtraTrees usan `max_features='sqrt'`, o sea que en cada division del arbol sortean un subconjunto de variables. **Las copias correlacionadas aumentan la probabilidad de que una variable util entre en el sorteo.** Para un ensamble de bagging, la redundancia no es ruido: es muestreo.

Es un buen ejemplo para la exposicion de que una practica estandar puede ser contraproducente segun el modelo, y de que conviene medir antes de aplicar recetas.

Informacion mutua

La informacion mutua mide cuanta informacion aporta una variable sobre la etiqueta, capturando relaciones no lineales — a diferencia de la correlacion, que solo ve las lineales. El notebook la calcula de **tres formas distintas**, y esa es la parte inteligente:

- **Sano contra falla:** que variables detectan que *algo* anda mal.
- **Los 14 estados:** que variables identifican *cual* es la falla.
- **La familia, mirando solo las ventanas con falla:** que variables separan las cuatro familias entre si. Esta es la que mas importa, porque es la que paga el 70 % del puntaje.

Con eso define una **ganancia de especificidad**: variables que pierden poca senal para detectar anomalia pero ganan importancia para identificar el estado. Son las candidatas a aportar de verdad.

Tamano del efecto por falla

Para cada una de las 13 fallas calcula dos medidas de cuanto separa cada variable esa falla del resto:

- **g de Hedges:** diferencia de medias dividida por el desvio combinado, con una correccion para muestras chicas. Dice cuantos desvios separan a los dos grupos.
- **delta de Cliff:** una medida no parametrica, basada en rangos, que no asume ninguna forma de distribucion. Util justamente porque 54 variables estan muy sesgadas.

El valor de esto para la exposicion es que permite decir **que variable delata a cada falla**, en lugar de hablar de importancia global.

Auditoria de las variables fisicas

Cruza, para cada una de las 42 fisicas, su informacion mutua en los tres escenarios contra su redundancia con las originales. Responde la pregunta que hay que hacerse siempre despues de inventar variables: **¿estas 42 aportan algo, o son re combinaciones sin informacion nueva?**

Comparacion de importancia entre modelos

Extrae la importancia de variables de cuatro modelos y la compara contra el ranking de informacion mutua, con dos medidas:

	Spearman vs MI_14	Jaccard top-20
LightGBM	0,764	0,111
RandomForest	0,712	0,026
CatBoost	0,734	0,026
RegLogistica	0,572	0,000

Como leerlo, y es un hallazgo lindo: la correlacion de Spearman es alta (0,57 a 0,76), o sea que los modelos *ordenan* las variables parecido a la informacion mutua. Pero el indice de Jaccard sobre las 20 mas importantes es casi cero: **casi no comparten las variables del podio**. Cada modelo se apoya en variables distintas aunque el ranking general se parezca.

Eso **explica por que el ensamble funciona**. No es magia estadistica: los modelos miran cosas distintas, y la regresion logistica es la que mas se aparta (Spearman 0,572, Jaccard 0,000). Es la justificacion cuantitativa de por que el peor modelo individual mejora el conjunto.

La seccion de datos NPZ

Es el arranque de la siguiente etapa, todavia exploratorio. Baja el paquete de senales crudas (664 MB) y establece los hechos basicos:

```
Total ventanas mapeadas: 4032      (1750 train + 1848 test + 434 sin etiquetar)
Claves por archivo NPZ: 21         (7 ventanas x 3 canales)
Formato: __

Acc      : 40000 muestras  (2,00 s a 20 kHz)
Current  : 20000 muestras  (2,00 s a 10 kHz)
Pressure :   400 muestras  (2,00 s a 200 Hz)
```

Y grafica comparaciones entre una maquina sana y una con falla, en el dominio del tiempo y con densidad espectral de potencia: aceleracion contra F01 (desbalance), corriente contra F08 (desequilibrio de tension) y presion contra F12 (cavitacion).

Estado honesto de esta seccion: es exploracion. Todavia no hay variables extraidas de las senales crudas dentro del modelo, ni por lo tanto un puntaje asociado. Es la base para la etapa siguiente.

Parte 5 — Los dos notebooks, comparados

	Intento3	base4.5
Celdas	62	89
Variables	174 (45 + 42 + 87)	174, las mismas
Modelo	LGB + RF + RegLogistica	el mismo
Validacion cruzada	51,98	51,98
Puntaje del servidor	48,1150	no se envio
EDA	basico: distribuciones y correlacion	profundo: calidad, redundancia, informacion mutua, tamano del efecto
Senales crudas	no	exploracion inicial
Que aporta	el resultado	la justificacion y el camino que sigue

La forma honesta de presentar la relacion entre los dos

Intento3 consiguio el numero. base4.5 explica por que ese numero es el que es y prepara lo que viene.

Eso no es un problema para la exposicion, al contrario: la consigna evalua las decisiones y su justificacion, no la cifra. Un notebook que documenta por que las variables fisicas aportan, por que el ensamble funciona y por que limpiar redundancia seria un error, vale para lo que se califica.

Parte 6 — Guion de exposicion

Una estructura posible en siete tramos. Los numeros en negrita son los que conviene decir de memoria.

1 El problema, en dos frases

Mantenimiento predictivo sobre maquinas rotativas: 13 modos de falla en cuatro familias. Pero no alcanza con clasificar, porque la metrica mide el **costo economico de la accion de mantenimiento** que el sistema toma a partir de nuestras probabilidades.

2 La metrica, y las dos cosas que descubrimos leyendola

Tres componentes: costo 70 %, calibracion 20 %, diagnostico 10 %. Primero: **el techo real es 73,96 y no 100**, porque aun acertando todo hay que pagar la inspeccion de 4 dolares. Segundo: **el 70 % paga por acertar la familia**, no la falla exacta — confundir F03 con F04 no cuesta nada. Ahi se decide donde poner el esfuerzo.

3 Los datos dicen algo distinto de la consigna

La consigna plantea un problema multilabel con fallas simultaneas. En las 1750 filas etiquetadas **no hay ninguna ventana con dos fallas**. Lo modelamos como 14 estados excluyentes con softmax. Ademas cada sesion tiene 7 ventanas con la misma etiqueta: el **tamano efectivo es 250, no 1750**.

4 El obstaculo real: train y test no comparten maquinas

26 maquinas contra 22, interseccion vacia, y ademas otro regimen: **1742 a 2052 rpm**. Lo atacamos por dos lados. Uno, normalizando cada variable contra su propia maquina: el KS medio baja de **0,139 a 0,059** y las 11 variables desalineadas quedan en cero. Dos, construyendo variables adimensionales.

5 Las variables fisicas, que fueron la palanca

42 variables construidas con criterio fisico y todas relativas: la vibracion dividida por el cuadrado de las rpm porque la energia escala asi; el cociente $2x/1x$ porque la desalineacion excita el segundo armonico; las temperaturas siempre contra el ambiente; las bombas por leyes de afinidad. Resultado: el ensamble pasa de **47,33 a 51,98**.

6 El modelo: por que un ensamble y no el mejor

Probamos seis algoritmos y **los seis caen en 2,3 puntos**, menos que el desvio entre particiones. El algoritmo no es la palanca. Pero ninguno gana en las tres componentes: los boosters discriminan mejor, los de bagging calibran mejor. El promedio simple captura las dos cosas. Lo llamativo: **la regresion logistica es el peor modelo individual y es la que hace funcionar el ensamble**, porque es la unica lineal y sus errores estan descorrelacionados. En base4.5 lo cuantificamos: los modelos comparten casi ninguna variable en su top 20 (Jaccard casi cero).

7 Resultado y que sigue

48,1150 en el servidor, contra un baseline de 29 y un techo de 74: estamos en dos tercios de lo alcanzable. Lo que queda es la senal cruda, y sabemos exactamente donde apuntarla: la exactitud de familia esta en **0,68** y ahi es donde el `cost_score` tiene los 25 puntos que le faltan.

Cuatro numeros para no olvidarse

48,1150 — lo que sacamos en el servidor. **29 / 30,78** — el baseline a superar. **73,96** — el techo real del problema. **0,68** — la exactitud de familia, que es el cuello de botella.

Parte 7 — Preguntas probables y como contestarlas

¿Por que 48 y no mas cerca de 100?

Porque 100 es inalcanzable por diseno de la metrica. Aun con prediccion perfecta hay que pagar la inspeccion de 4 dolares en cada ventana con falla, y eso deja el `cost_score` maximo en 62,8 y el final en 73,96. Nuestro 48 sobre 74 es el **65 % de lo alcanzable**.

La consigna dice multilabel, ¿por que usan softmax?

Porque los datos etiquetados no tienen ni un solo caso de falla combinada: `is_combo` vale cero en las 1750 filas y cada ventana tiene exactamente cero o una falla. El softmax aprovecha esa restriccion y devuelve probabilidades coherentes entre si. **Es un riesgo asumido y conocido**: si el test tuviera combinaciones, el softmax las subestimaria. La cobertura seria mezclar una rama de 13 clasificadores binarios.

¿Como evitan que el modelo haga trampa con la validacion?

Agrupando por maquina. Si una maquina apareciera de los dos lados de la particion, el modelo la reconoceria y el resultado seria una mentira. Usamos `StratifiedGroupKFold`, que ademas mantiene la proporcion de estados en cada particion. Y nunca usamos el test para entrenar ni ajustar.

¿Por que la validacion da 51,98 y el servidor 48,11?

Porque la validacion cruzada agrupada por maquina mide *maquina nueva, mismo regimen*, y el test es *maquina nueva, regimen nuevo*. Es una brecha esperable y la tenemos identificada. Por eso el notebook incluye una segunda validacion que entrena con las 13 maquinas lentas y valida con las 13 rapidas, que es el regimen del test.

¿Por que no probaron redes neuronales?

Por el tamano efectivo de la muestra: 250 sesiones, no 1750 ventanas. Una red profunda memorizaria las 26 maquinas en lugar de aprender el patron de falla. A eso se suma que los tres canales de senal cruda tienen frecuencias de muestreo distintas (20 kHz, 10 kHz y 200 Hz), lo que complica armar tensores unificados, y que el cambio de regimen castiga especialmente a los modelos profundos.

Si el ensamble incluye el peor modelo, ¿no es arbitrario?

No, y lo medimos. En base4.5 comparamos la importancia de variables entre modelos: la correlacion de rankings es alta (0,57 a 0,76) pero el solapamiento del top 20 es casi nulo. **Cada modelo se apoya en variables distintas**. La regresion logistica es la que mas se aparta, y por eso aporta al promedio aunque sola sea la peor.

¿Que harian con mas tiempo?

Extraer variables de las senales crudas, apuntando a dos cosas concretas: el espectro de **envolvente** en la banda resonante, que es lo unico que separa los tres tipos de falla de rodamiento entre si, y las **bandas laterales de la corriente**, que delatan barras rotoricas. Con el mapa que tenemos, mejorar la exactitud de familia es lo que mas paga.

¿No convenia limpiar las variables redundantes?

Lo probamos y **empeora**. Quitar las 12 columnas mas correlacionadas hizo bajar el resultado de 45,44 a 44,45. `RandomForest` y `ExtraTrees` sortean un subconjunto de variables en cada division: las copias correlacionadas aumentan la probabilidad de que una variable util entre en el sorteo. Para un ensamble de bagging la redundancia funciona como muestreo.

Parte 8 — Glosario

Terminos que aparecen en los notebooks y conviene poder explicar si preguntan.

Termino	Que significa
Ventana	Fragmento de 2 segundos de senal del que se calculan todos los descriptores. Es la unidad que se clasifica.
Sesion	Conjunto de 7 ventanas consecutivas de la misma maquina. Comparten etiqueta.
OOF (out-of-fold)	Prediccion que un modelo hace sobre datos que no vio al entrenar. Es la unica estimacion honesta de como se va a comportar.
StratifiedGroupKFold	Particion cruzada que mantiene la proporcion de clases (estratificada) y no parte los grupos (una maquina entera cae de un solo lado).
Covariate shift	Que las variables de entrada tengan distinta distribucion en entrenamiento y en test, aunque la relacion con la etiqueta sea la misma. Es nuestro caso: otro regimen de rpm.
Test de Kolmogorov-Smirnov	Mide cuanto difieren dos distribuciones. Va de 0 (identicas) a 1. Lo usamos para ver si la normalizacion alinea train y test.
Z-score por maquina	Restar la mediana de la maquina y dividir por su desvio. Convierte el valor absoluto en desviacion respecto del comportamiento propio.
Brier	Promedio de (probabilidad - verdad) ² . Mide calibracion: si decis 0,7 muchas veces, deberia pasar el 70 % de las veces.
F1 macro	Promedio del F1 de cada clase dandoles el mismo peso, incluidas las raras. Con umbral 0,5 fijo.
Softmax	Capa que produce probabilidades que suman 1 sobre clases mutuamente excluyentes.
Ensamble	Promediar las predicciones de varios modelos. Funciona cuando los modelos se equivocan en cosas distintas.
Bagging / boosting	Bagging entrena muchos modelos independientes y promedia (RandomForest, ExtraTrees): calibra bien. Boosting entrena en cadena corrigiendo errores (LightGBM, XGBoost, CatBoost): discrimina mejor.
Informacion mutua	Cuanta informacion aporta una variable sobre la etiqueta, incluyendo relaciones no lineales.
g de Hedges	Tamano del efecto: cuantos desvios separan las medias de dos grupos, con correccion para muestras chicas.
delta de Cliff	Tamano del efecto no parametrico, basado en rangos. No asume ninguna forma de distribucion.
Jaccard	Proporcion de elementos compartidos entre dos conjuntos. Lo usamos para ver cuanto se solapan los top 20 de variables de cada modelo.
1x, 2x	Amplitud de la vibracion a la frecuencia de giro y al doble. El desbalance carga el 1x; la desalineacion excita el 2x.
Curtosis y factor de cresta	Miden cuan puntiaguda es la senal. Suben cuando hay impactos, tipicos de danos incipientes en rodamientos.
Leyes de afinidad	Relaciones de las bombas: el caudal escala con las rpm y la altura con las rpm al cuadrado. Permiten construir indicadores adimensionales.
Envolvente	Contorno de amplitud de la senal. Su espectro revela las frecuencias de defecto de rodamiento, que no se ven en el espectro directo.

Advertencia de honestidad para la exposicion

Todos los numeros de esta guia salen de las salidas guardadas en los dos notebooks. Dos aclaraciones que conviene no ocultar si preguntan:

base4.5 no tiene puntaje de servidor propio: no se envio. Su seccion de senales crudas es exploratoria y todavia no alimenta el modelo.

La comparacion entre validacion y servidor tiene una sola referencia solida (51,98 contra 48,1150). Con un solo punto de calibracion no se puede prometer cuanto va a dar una configuracion nueva, solo estimar.